

EBU6304 Software Engineering Group Project

2025-26

Final short report

Group number	Group-02	Final submission
member 1	QM no: 231221386	Name: Zhou Jinglun
member 2	QM no: 231221320	Name: Wang Dawei
member 3	QM no: 231221261	Name: Lin Kunpeng
member 4	QM no: 231221711	Name: Yan Wutong
member 5	QM no: 231222246	Name: Jiang Yanzhen
member 6	QM no: 231222855	Name: Tao Xingyu

Design

The system is designed as a lightweight Servlet/JSP web application for the teaching-assistant recruitment workflow. The final architecture uses four clear layers. The controller layer contains page routing and REST-style API servlets. The service layer holds business rules such as role validation, duplicate-application prevention, workload checks, and AI orchestration. The entity layer defines the core domain objects, including users, jobs, and applications. The storage layer persists those entities as JSON files and uploaded documents under the local data directory. Utility classes support JSON response envelopes, path resolution, and shared validation helpers.

The main design principle is separation of responsibility. JSP and JavaScript handle presentation and interaction only; controllers translate HTTP requests; services enforce workflow rules; storage only reads and writes files. This avoids mixing UI concerns with persistence logic and makes the code easier to test. The project intentionally avoids a database because the coursework requires simple file-based input and output. The data model therefore uses stable IDs such as TA001, MO001, JOB001, and APP001 so records can be linked across users, jobs, applications, and uploaded CV files.

The user experience is organised around three role workspaces. TAs maintain profiles and CVs, browse jobs, apply, and track decisions. Module Organisers create jobs, inspect applicants, view candidate CVs, request AI summaries, and record selection decisions. Administrators monitor users, applications, and workload risk. The AI assistant is treated as a supporting decision tool rather than the source of truth: deterministic system data is prepared first, then the model is asked to summarise or prioritise it. This keeps the recommendations explainable and prevents the model from inventing system state.

Implementation

The project is implemented with Java Servlet/JSP, HTML, CSS, and JavaScript, packaged as a Maven WAR and deployed to Tomcat 10.1. Maven controls compilation, dependency management, test execution, and WAR generation. The Java source is compiled to a Java 17 target and can be built on the group development machines using JDK 21. Runtime data is stored under the project data directory by default, with users.json, jobs.json, applications.json, and an uploads folder for TA CV files. The data directory can also be overridden with TARS_DATA_DIR for deployment flexibility.

The frontend was first built as a full-role interface and then connected to real servlet APIs. Mock runtime files were removed before finalisation so the application now uses the real API path only. The login and registration pages support role selection, TA-only CV upload, and server-side session authentication. The dashboard pages load live JSON-backed data through the frontend API adapter. Secure CV access is handled through a controlled file servlet instead of exposing arbitrary filesystem paths. The AI provider is configured through environment variables rather than hardcoded secrets, so each developer can supply a local key without committing it to Git.

Version control was used to structure the project across sprints. Early commits established requirements, architecture, prototype screens, and report artefacts. Sprint 2 introduced the Servlet/JSP skeleton, role pages, frontend interaction, and API documentation. Later commits added the real backend services, JSON storage,

four-layer architecture, CV access, AI assistant tools, GLM/Z.AI provider integration, dataset finalisation, documentation cleanup, and regression tests. Feature branches were used for member work, with integration into shared sprint branches after review.

Testing

The testing strategy combines automated service-layer tests, API smoke validation, data integrity checks, and manual acceptance testing. Automated tests focus on business rules because those rules are the highest-risk part of the system: authentication, role permissions, duplicate application prevention, MO access control, status updates, workload limits, CV validation, AI fallback behaviour, and final demo dataset consistency. The current automated suite contains 44 JUnit tests, all passing during the final build.

Specific examples include rejecting duplicate TA applications with `APPLICATION_DUPLICATE`, preventing a Module Organiser from reading applicants for a job owned by another MO, blocking candidate selection when projected TA workload exceeds the safe threshold, rejecting invalid CV extensions, and confirming that AI endpoints still return structured fallback output when no provider key is configured. The final dataset integrity test also checks that the demonstration data contains at least 50 TA accounts, 20 MO accounts, a valid Admin account, two jobs per MO, representative application statuses, and CV files only for TA users.

Manual testing is still needed for browser-level acceptance. The project includes a final manual acceptance checklist covering login, registration, TA profile and CV flow, TA job application, MO job management, MO candidate review, Admin monitoring, AI assistant actions, error handling, and UI stability. API validation is supported by a smoke-test plan and script that checks representative endpoints after Tomcat deployment. This layered testing approach is practical for the project size: unit tests protect business rules, smoke tests protect integration, and manual checks protect visible workflows.

The use of Generative AI (GenAI)

During the development process, GenAI tools were used as supporting tools rather than as a replacement for our own design and implementation work. We used GenAI mainly for brainstorming alternative workflow ideas, checking whether the requirements had been interpreted consistently, improving the wording and structure of documentation, and generating possible test scenarios for discussion. For example, when preparing requirement analysis, API contracts, and final documentation, GenAI helped us identify unclear wording and suggest a more professional structure. The final content was still reviewed and adjusted by the group to match the actual system and coursework requirements.

GenAI was also useful during frontend and user-experience refinement. We used it to compare different interface layouts, improve English interface text, suggest more polished visual styles, and review whether pages were consistent across TA, MO, and Admin roles. It also helped us think through edge cases such as invalid login, duplicate applications, CV upload errors, role permissions, and workload warnings. These suggestions were treated as references only; the group still implemented, tested, and corrected the software manually. In this sense, GenAI improved efficiency in planning, writing, reviewing, and polishing, but the project decisions and final code remained under group control.

The limitations of GenAI were clear. It sometimes produced general suggestions that did not fit the Servlet/JSP technology route or the file-based storage constraint. It could also describe features more confidently than the actual implementation supported, so we had to verify every suggestion against the code, the running system, and the project handout. We therefore used GenAI most effectively as a reviewer and assistant: helpful for idea generation, wording improvement, UI refinement, and test planning, but not reliable enough to be accepted without human checking.

Individual contribution and reflection

Zhou Jinglun

Main contribution: Zhou Jinglun acted as the main integrator and technical lead, and was the largest contributor to the project. He set up the project structure, organised the Sprint 1 documentation baseline, developed the main Servlet/JSP frontend, refined the UI style and interaction design, implemented the CV upload flow, migrated runtime storage to local JSON data files, added service-layer tests, aligned the code with the four-layer architecture, implemented secure CV access, integrated the AI assistant features, prepared final demonstration data, and cleaned up the final documentation set. He also coordinated branch integration, prepared the final report, and kept the implementation aligned with the coursework constraints.

Reflective statement: This project strengthened my ability to manage a software engineering task as a complete lifecycle rather than only a coding exercise. The most difficult part was keeping architecture,

documentation, frontend behaviour, backend services, data storage, and version control consistent while requirements changed. I improved my understanding of Servlet/JSP deployment, JSON persistence, role-based workflows, AI integration, and practical project leadership. I also learned that final delivery quality depends heavily on testing, cleanup, and clear documentation, not only on adding new features.

Wang Dawei

Main contribution: Wang Dawei contributed to the early prototype and interface work, including TA dashboard and job-listing visual material. He also supported later documentation work by preparing API synchronisation notes and writing the final user manual. His work helped connect the initial UI ideas with the later implemented system and provided user-facing guidance for running and understanding the software.

Reflective statement: Through the project, I gained a clearer understanding of how early prototypes influence later implementation. A prototype is useful only if it communicates the intended workflow and can later be mapped to real screens and data. I also learned how documentation supports collaboration, especially when frontend and backend work are developed by different people. The process helped me improve my ability to describe technical behaviour clearly and to connect visual design with implemented functionality.

Lin Kunpeng

Main contribution: Lin Kunpeng contributed Sprint 1 prototype screens, implemented API servlet work in Sprint 2, added validation for Module Organiser job and selection rules in Sprint 3-4, and contributed Sprint 4 service regression testing. He also took responsibility for recording and editing the final demonstration video, which helped present the completed system in a clear and structured way.

Reflective statement: This project helped me understand how backend API design affects the whole application. Implementing servlets and validation rules required careful thinking about permissions, request formats, and error responses. I also learned that tests are not just a final task; they clarify expected behaviour and prevent accidental regressions. The main challenge was keeping the implementation consistent with the role rules, especially for MO permissions and candidate selection logic.

Yan Wutong

Main contribution: Yan Wutong contributed Sprint 1 report material and MO/Admin dashboard prototype work. He later worked on Admin job/application filtering, API smoke validation, final testing documentation, and the final Javadoc documentation. He also helped design the structure and script for the final demonstration video. His contribution improved both the Admin workflow and the technical documentation prepared for final submission.

Reflective statement: I learned how important it is to make testing and documentation concrete enough for another person to repeat. API smoke testing forced me to think about the system from the outside: login, session cookies, response envelopes, role restrictions, and expected failures. I also developed a better understanding of how Admin functions differ from TA and MO workflows. The project improved my ability to connect requirements, endpoints, and visible application behaviour.

Jiang Yanzen

Main contribution: Jiang Yanzen contributed to the Sprint 1 Product Backlog and requirements analysis, added service interface files during Sprint 2, and prepared Sprint 3-4 completion and integration documentation. She also helped design the final demonstration video structure and script. Her work helped define the required system behaviour, support implementation planning, and prepare the group for final presentation.

Reflective statement: This project improved my understanding of how requirement documents and service contracts guide later implementation. Writing backlog and requirement material required us to define what the system should do before coding. Service interfaces also showed the value of separating what a module promises from how it is implemented. The biggest challenge was keeping documentation accurate as the software evolved, but this also helped me learn how agile projects require continuous alignment.

Tao Xingyu

Main contribution: Tao Xingyu contributed early job-application and job-detail prototype work, supported storage and model improvements, prepared integration test evidence, and updated Sprint 4 architecture, storage, and AI-related documentation. He also helped design the final demonstration video structure and script. His work helped connect the prototype, data model, technical documentation, and final presentation of the system.

Reflective statement: I learned that small backend and documentation changes can have a large impact on whether the project is understandable and maintainable. Working on model/storage and integration evidence helped me see how data structures, files, and service behaviour connect to user workflows. I also learned the importance of using branches correctly and keeping local work up to date before merging. The project improved my awareness of collaboration discipline in Git-based teamwork.